

Python Developer Deep-Dive Interview Q&A

Primary: Python · REST API · Microservices · TDD

Good to have: Apache Camel · Kafka · Couchbase · Oracle DB · FastAPI

Additional: Airflow · AWS Lambda · Threading · Memory Mgmt · Migration · Terraform · Databricks

- | | |
|--|---|
| 1. Python — Basic to Intermediate | 11. Apache Kafka |
| 2. Python — Advanced Concepts | 12. Couchbase |
| 3. Exception Handling in Python | 13. Airflow DAGs & Tasks |
| 4. Memory Management in Python | 14. AWS Lambda & Orchestration |
| 5. Multithreading & Multiprocessing (Python vs Java) | 15. Streaming Large Files in AWS Lambda |
| 6. Synchronization in Python | 16. Java/Scala → Python Migration |
| 7. TDD — Unit & Integration Testing | 17. Terraform & Databricks |
| 8. FastAPI | 18. Microservices & Team Leadership |
| 9. REST API | 19. Coding Challenge: Anagram Grouping |
| 10. Apache Camel | |

1. Python — Basic to Intermediate

1. What are Python's key features that make it ideal for backend and data engineering?

Dynamic typing, clean readable syntax, an enormous standard library, first-class functions, and a vast ecosystem (Django, FastAPI, Pandas, Spark PySpark, Boto3). It is interpreted (fast iteration), supports multiple paradigms (OOP, functional, procedural), and has strong community backing. For backend/data work: strong REST frameworks, excellent async support via asyncio, and seamless integration with AWS services.

2. Mutable vs immutable types — why does it matter?

Immutable: int, float, str, tuple, frozenset — cannot change after creation; safe to use as dict keys, safe to share across threads. **Mutable:** list, dict, set, bytearray — can be changed in place; watch out for aliasing bugs.

```
# aliasing bug with mutable default
def bad(x, items=[]): # same list reused every call!
    items.append(x); return items
def good(x, items=None):
    if items is None: items = []
    items.append(x); return items
```

3. Explain list comprehension vs generator expression vs map/filter

```
# list comprehension – builds full list in memory
squares = [x*x for x in range(1000)]

# generator expression – lazy, one item at a time (memory-efficient)
squares_gen = (x*x for x in range(1000))

# map/filter – functional style, also lazy in Python 3
squares_map = map(lambda x: x*x, range(1000))
evens = filter(lambda x: x%2==0, range(1000))
```

For large datasets, always prefer generators over list comprehensions to avoid loading everything into RAM.

4. *args and **kwargs — explain with a real example

```
def log_event(event_type, *messages, **metadata):
    # *messages: positional extras as tuple
    # **metadata: keyword extras as dict
    print(f"[{event_type}]", " | ".join(messages))
    print("Meta:", metadata)

log_event("ERROR", "DB timeout", "retry #3",
          service="order", region="us-east-1")
# [ERROR] DB timeout | retry #3
# Meta: {'service': 'order', 'region': 'us-east-1'}
```

5. What are Python decorators? Provide a real-time example

A decorator is a higher-order function that wraps another function to add behavior without modifying it. The `@decorator` syntax is sugar for `func = decorator(func)`. Real-time examples: auth checks, logging, retry logic, caching, timing.

```
import functools, time, logging

def retry(max_tries=3, delay=1.0):
    """Real-world: retry a flaky external API call."""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, max_tries+1):
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    logging.warning(f"Attempt {attempt} failed: {e}")
                    if attempt == max_tries: raise
                    time.sleep(delay)
            return wrapper
        return decorator

@retry(max_tries=3, delay=2.0)
def call_payment_api(order_id):
    # may raise requests.Timeout
    return requests.post(f"/pay/{order_id}", timeout=5)
```

Always use `@functools.wraps(func)` so the wrapper preserves `__name__` and `__doc__` — critical for debugging, introspection, and Sphinx docs.

6. What is the difference between deep copy and shallow copy?

```
import copy
original = [[1,2],[3,4]]

shallow = copy.copy(original)
shallow[0].append(99)
print(original)  # [[1,2,99],[3,4]] – inner list shared!

deep = copy.deepcopy(original)
deep[0].append(88)
print(original)  # [[1,2,99],[3,4]] – unchanged
```

7. How does Python handle variable scoping? (LEGB rule)

Python resolves names in four scopes in order: **Local** → **Enclosing** (closures) → **Global** (module) → **Built-in**. Use `global` to assign to a module-level name, and `nonlocal` to assign to an enclosing-function name.

```
x = "global"
def outer():
    x = "enclosing"
    def inner():
        nonlocal x
        x = "changed enclosing"
    inner(); print(x)  # changed enclosing
outer(); print(x)     # global (unchanged)
```

8. What is a lambda function? When should you NOT use it?

An anonymous single-expression function. Use for short inline operations passed to `sorted`, `map`, `filter`. Avoid when: the logic is complex (unreadable), you need a docstring, or you're using it more than once (define a named function instead).

```
users = [{"name": "Ravi", "age": 30}, {"name": "Asha", "age": 25}]
sorted_users = sorted(users, key=lambda u: u["age"])
```

9. Difference between `is` and `==` ?

`==` compares values (equality); `is` compares object identity (same memory address). Only use `is` for singletons: `None`, `True`, `False`. Integer caching (-128 to 256) makes `is` seem to work for small ints — never rely on it.

10. Explain Python's data model — __dunder__ methods

```
class Money:
    def __init__(self, amount, currency="INR"):
        self.amount, self.currency = amount, currency
    def __repr__(self): return f"Money({self.amount} {self.currency})"
    def __add__(self, other): return Money(self.amount+other.amount, self.currency)
    def __eq__(self, other): return self.amount==other.amount and self.currency==other.currency
    def __lt__(self, other): return self.amount < other.amount

a = Money(100); b = Money(200)
print(a+b)      # Money(300 INR)
print(a < b)    # True
```

11. What are context managers and the with statement?

Objects implementing `__enter__` / `__exit__` ensure guaranteed cleanup even on exceptions. Use `@contextmanager` for simple cases.

```
from contextlib import contextmanager

@contextmanager
def db_transaction(conn):
    try:
        yield conn
        conn.commit()
    except Exception:
        conn.rollback(); raise
    finally:
        conn.close()

with db_transaction(get_conn()) as conn:
    conn.execute("INSERT INTO orders ...")
```

12. Explain generators and the yield keyword

```
def read_large_csv(filepath):
    """Stream rows without loading whole file into RAM."""
    with open(filepath) as f:
        header = next(f)
        for line in f:
            yield line.strip().split(",")

for row in read_large_csv("orders.csv"):
    process(row)  # only one row in memory at a time
```

`yield from` delegates to a sub-generator (used in async coroutines too).

13. What are Python's built-in data structures and when to use each?

Structure	Best for	Lookup	Insert
list	Ordered sequence, stacks	O(n)	O(1) end, O(n) middle
dict	Key-value, fast lookup	O(1) avg	O(1) avg
set	Uniqueness, membership test	O(1) avg	O(1) avg
tuple	Immutable records, dict keys	O(n)	N/A
collections.deque	Queue/deque, fast both ends	O(n)	O(1) both ends
heapq	Priority queue	O(log n)	O(log n)

14. What is the GIL? How does it affect Python programs?

The Global Interpreter Lock is a mutex in CPython that allows only one thread to execute Python bytecode at a time.

Impact: CPU-bound threads don't gain from multiple cores; I/O-bound threads are fine (GIL released during I/O).

Solutions: `multiprocessing` for CPU-bound work (separate processes, separate GILs), async I/O for high-concurrency I/O, or C extensions that release the GIL.

15. What are class methods, static methods, and instance methods?

```
class User:
    count = 0
    def __init__(self, name): self.name = name; User.count += 1

    def greet(self):          # instance method – has self
        return f"Hi, {self.name}"

    @classmethod
    def from_dict(cls, d):    # class method – factory pattern
        return cls(d["name"])

    @staticmethod
    def validate_name(n):     # static method – no self/cls
        return bool(n and n.isalpha())
```

16. How does inheritance and MRO work in Python?

Python supports multiple inheritance; Method Resolution Order (C3 linearization) defines which parent's method is called. Use `super()` to call the next in MRO. Check MRO with `ClassName.__mro__`.

```
class A:
    def speak(self): return "A"
class B(A):
    def speak(self): return "B-" + super().speak()
class C(A):
    def speak(self): return "C-" + super().speak()
class D(B, C): pass
print(D().speak())      # B-C-A    (MRO: D→B→C→A)
```

17. What are abstract base classes (ABCs)?

```
from abc import ABC, abstractmethod

class DataConnector(ABC):
    @abstractmethod
    def connect(self): ...
    @abstractmethod
    def fetch(self, query): ...

class PostgresConnector(DataConnector):
    def connect(self): return psycopg2.connect(...)
    def fetch(self, query): ... # must implement or TypeError
```

18. What is a namedtuple and dataclass?

```
from collections import namedtuple
from dataclasses import dataclass, field

Point = namedtuple("Point", ["x","y"]) # immutable, lightweight

@dataclass
class Order:
    id: int
    items: list = field(default_factory=list)
    total: float = 0.0
    # auto __init__, __repr__, __eq__ generated
```

dataclass is the modern choice for data-carrying objects (DTOs). For truly immutable value objects, use `@dataclass(frozen=True)`.

19. Explain Python type hints and mypy

```
from typing import Optional, List, Dict, Union, TypeVar

def get_user(user_id: int) -> Optional[Dict[str,str]]:
    ...

T = TypeVar("T")
def first(lst: List[T]) -> Optional[T]:
    return lst[0] if lst else None

# Python 3.10+ – union with pipe
def process(value: int | str | None) -> str: ...
```

20. What is the difference between `__str__` and `__repr__`?

`__repr__` = unambiguous machine-readable representation (used in debugger, logs, REPL). `__str__` = human-friendly string (used by `print()`, `str()`). Rule: `repr` should ideally be valid Python to recreate the object; `str` can be pretty.

21. What is a Python module vs package vs namespace package?

A **module** is a single `.py` file. A **package** is a directory with `__init__.py` that groups modules. A **namespace package** (PEP 420) has no `__init__.py` and can span multiple directories — useful for large plugin systems.

22. How does Python's import system work? Circular imports pitfall

Python caches imported modules in `sys.modules`. Circular imports (A imports B, B imports A) cause `AttributeError` or `ImportError`. Fix by moving the import inside the function, restructuring into a third module, or using `importlib`.

23. What is pickling? Use cases and security risks

```
import pickle
data = {"model": trained_model, "features": feature_list}
with open("model.pkl", "wb") as f:
    pickle.dump(data, f) # serialize
# DANGER: never unpickle untrusted data – RCE risk
# Alternative: JSON for data, joblib/safetensors for models
```

24. Explain slots in Python classes

```
class Event:
    __slots__ = ["id", "name", "ts"]
    def __init__(self, id, name, ts):
        self.id=id; self.name=name; self.ts=ts
# Benefit: no per-instance __dict__ → ~40% less memory,
#         faster attribute access. Use for high-count value objects.
```

25. What is functools and its key utilities?

```
from functools import lru_cache, partial, reduce, cached_property

@lru_cache(maxsize=256)
def fib(n): return n if n < 2 else fib(n-1)+fib(n-2)

add5 = partial(lambda x,y: x+y, 5) # partial application
total = reduce(lambda a,b: a+b, [1,2,3,4]) # fold

class Config:
    @cached_property
    def settings(self): return load_from_env() # computed once, then cached
```

2. Python — Advanced Concepts

1. Async/await and the event loop — how does it work?

Python's asyncio provides a single-threaded event loop that runs coroutines cooperatively. When a coroutine hits `await`, it suspends and the event loop can run another coroutine. This is ideal for high-concurrency I/O (HTTP calls, DB queries) — thousands of concurrent operations on one thread, no GIL overhead.

```
import asyncio, httpx

async def fetch(client, url):
    resp = await client.get(url)
    return resp.json()

async def main():
    async with httpx.AsyncClient() as client:
        # Fire all requests concurrently, not sequentially
        results = await asyncio.gather(
            fetch(client, "https://api/orders"),
            fetch(client, "https://api/users"),
            fetch(client, "https://api/inventory"),
        )
    print(results)

asyncio.run(main())
```

Advanced: `asyncio.TaskGroup` (Python 3.11+) provides structured concurrency with automatic cancellation on failure. Use `asyncio.Queue` for producer-consumer patterns in async code.

2. Metaclasses — what they are and when to use them

A metaclass is the class of a class — it controls how a class is created. `type` is the default metaclass. Use cases: ORM frameworks (Django's Model), API frameworks that auto-register routes, enforcing coding standards, or singleton patterns.

```
class SingletonMeta(type):
    _instances = {}
    def __call__(cls, *a, **kw):
        if cls not in cls._instances:
            cls._instances[cls] = super().__call__(*a, **kw)
        return cls._instances[cls]

class Config(metaclass=SingletonMeta):
    def __init__(self): self.db_url = os.getenv("DB_URL")

c1, c2 = Config(), Config()
assert c1 is c2 # True — same instance
```

3. Descriptors — the machinery behind properties and ORMs

```
class Validated:
    def __set_name__(self, owner, name): self.name = name
    def __get__(self, obj, typ): return obj.__dict__.get(self.name)
    def __set__(self, obj, value):
        if not isinstance(value, str):
            raise TypeError(f"{self.name} must be str")
        obj.__dict__[self.name] = value

class User:
    name = Validated() # descriptor on the class
    email = Validated()
```

4. What is a closure? Real use case.

```
def make_rate_limiter(max_calls_per_minute):
    calls = []
    def allow():
        now = time.time()
        calls[:] = [t for t in calls if now-t < 60]
        if len(calls) < max_calls_per_minute:
            calls.append(now); return True
        return False
    return allow # captures 'calls' and 'max_calls'

limiter = make_rate_limiter(10)
if limiter(): process_request()
```

5. What is the Python data model's iterator protocol?

```
class Countdown:
    def __init__(self, n): self.n = n
    def __iter__(self): return self
    def __next__(self):
        if self.n <= 0: raise StopIteration
        self.n -= 1; return self.n+1

for n in Countdown(3): print(n) # 3 2 1
```

6. How does Python's garbage collection work?

Python uses **reference counting** as the primary mechanism — when an object's count reaches zero it's freed immediately. A **cyclic garbage collector** (the `gc` module, generational, three generations) handles reference cycles. `gc.collect()` forces a cycle collection. Weak references (`weakref`) let you reference an object without preventing collection.

7. What are protocol classes (structural subtyping)?

```
from typing import Protocol

class Serializable(Protocol):
    def to_dict(self) -> dict: ...

def save(obj: Serializable) -> None: # accepts any duck type
    db.insert(obj.to_dict())

# Any class with to_dict() passes – no explicit inheritance
```

8. What is the difference between @property, getter, setter, deleter?

```
class Temperature:
    def __init__(self, celsius): self._c = celsius
    @property
    def celsius(self): return self._c
    @celsius.setter
    def celsius(self, v):
        if v < -273.15: raise ValueError("Below absolute zero")
        self._c = v
    @property
    def fahrenheit(self): return self._c * 9/5 + 32
```

9. Advanced: How to profile Python code and find bottlenecks?

```
import cProfile, pstats

with cProfile.Profile() as pr:
    my_expensive_function()

stats = pstats.Stats(pr)
stats.sort_stats("cumulative").print_stats(10)

# Line profiler: pip install line_profiler
# @profile decorator on the function, run: kernprof -l script.py
# Memory: pip install memory_profiler → @memory_profiler.profile
```

10. Python 3.10–3.12 new features worth knowing

```
# Structural pattern matching (3.10)
match command.split():
    case ["quit"]: quit()
    case ["go", dir]: go(dir)
    case ["pick", item]: pick(item)
    case _: print("unknown")

# Exception groups (3.11)
try: ...
except* ValueError as eg: handle_group(eg)

# f-string debugging (3.8+)
x = 42; print(f"{x=}") # x=42
```

3. Exception Handling in Python

1. Core concepts — try/except/else/finally

```
def read_config(path):
    try:
        f = open(path)
    except FileNotFoundError as e:
        logging.error(f"Config missing: {e}")
        return {} # graceful default
    except (PermissionError, OSError) as e:
        raise RuntimeError("Cannot open config") from e # chain
    else:
        return json.load(f) # runs only if no exception
    finally:
        try: f.close() # always runs
        except: pass
```

Rule: catch the most specific exception first. Never use bare `except:` — it catches `SystemExit` and `KeyboardInterrupt` too.

2. Custom exception hierarchy — best practice

```
class AppError(Exception): # base for all app errors
    def __init__(self, msg, code=500):
        super().__init__(msg); self.code = code

class ValidationError(AppError):
    def __init__(self, field, msg):
        super().__init__(f"{field}: {msg}", code=422)
        self.field = field

class NotFoundError(AppError):
    def __init__(self, resource):
        super().__init__(f"{resource} not found", code=404)

# FastAPI / Flask: catch AppError at the global handler level
# and return the right HTTP status code
```

3. Exception chaining — raise from

```
try:
    result = requests.get(url, timeout=5)
    result.raise_for_status()
except requests.RequestException as e:
    raise ServiceUnavailableError("payment-svc") from e
# full traceback preserved, original error visible
```

4. Exception hierarchy in Python

`BaseException` → `SystemExit`, `KeyboardInterrupt`, `GeneratorExit`, and `Exception`. Under `Exception`: `ArithmeticError`, `LookupError`, `OSError`, `TypeError`, `ValueError`, `RuntimeError`, etc. Your custom exceptions should extend `Exception`, not `BaseException`.

5. Context managers for exception safety

```
class SuppressAndLog:
    def __init__(self, *exc_types): self.exc_types = exc_types
    def __enter__(self): return self
    def __exit__(self, exc_type, exc_val, tb):
        if exc_type and isinstance(exc_type, self.exc_types):
            logging.warning(f"Suppressed: {exc_val}")
            return True # True = suppress the exception

with SuppressAndLog(TimeoutError, ConnectionError):
    call_flaky_service()
```

6. Global exception handler in FastAPI

```
from fastapi import FastAPI, Request
from fastapi.responses import JSONResponse

app = FastAPI()

@app.exception_handler(AppError)
async def app_error_handler(req: Request, exc: AppError):
    return JSONResponse(
        status_code=exc.code,
        content={"error": str(exc), "path": str(req.url)}
    )

@app.exception_handler(Exception)
async def generic_handler(req: Request, exc: Exception):
    logging.exception("Unhandled error")
    return JSONResponse(status_code=500, content={"error": "Internal error"})
```

4. Memory Management in Python

1. How Python manages memory — the full picture

Python uses a private heap. The memory manager handles allocation; the **pymalloc** allocator optimizes for small objects (<512 bytes). Objects on the heap are tracked by reference count; the cyclic GC handles cycles. Python's memory is not returned to the OS immediately — it's pooled for reuse.

2. Reference counting in detail

```
import sys
a = []
print(sys.getrefcount(a)) # 2 (a + getrefcount arg)
b = a
print(sys.getrefcount(a)) # 3
del b
print(sys.getrefcount(a)) # 2
# When count hits 0, CPython frees the object immediately
```

3. Common memory leaks and how to detect/fix them

Cause	Detection	Fix
Reference cycles	<code>gc.get_objects()</code> , <code>tracemalloc</code>	<code>gc.collect()</code> , restructure refs
Growing caches with no eviction	Memory profiler over time	<code>@lru_cache(maxsize=N)</code> , explicit TTL
Closures holding large objects	<code>objgraph.show_most_common()</code>	Capture only what's needed
Open file/connection handles	resource module, file descriptors	Context managers (with)
Global lists/dicts accumulating	Heap dump	Bounded data structures (deque)

```
import tracemalloc
tracemalloc.start()
# ... run code ...
snapshot = tracemalloc.take_snapshot()
top = snapshot.statistics("lineno")
for stat in top[:5]: print(stat)
```

4. Weak references — preventing memory leaks in caches

```
import weakref

class Cache:
    def __init__(self): self._store = weakref.WeakValueDictionary()
    def put(self, k, v): self._store[k] = v
    def get(self, k): return self._store.get(k)
# Values are GC'd when no other strong reference exists — no leak
```

5. Memory optimization techniques for production

- Use `__slots__` for high-count objects (~40% RAM saving).
- Generators instead of lists for pipelines.
- NumPy arrays over Python lists for numerical data.
- Chunked DB reads (`fetchmany`) instead of `fetchall()` .
- Streaming S3/file reads with `boto3` 's `StreamingBody` .
- `array` module for homogeneous typed sequences.
- `sys.getsizeof()` to measure; `memory_profiler` to profile line-by-line.

6. Interning and object reuse

CPython interns small integers (-5 to 256) and short strings (identifiers). This is an implementation detail — never write code that relies on it. It does explain why `a = 1000; b = 1000; a is b` can be `False` .

5. Multithreading & Multiprocessing — Python vs Java

1. Complete comparison: threading models in Python vs Java

Aspect	Python	Java
Thread creation	<code>threading.Thread</code>	<code>Thread</code> , <code>Runnable</code> , <code>Callable</code>
Thread pools	<code>concurrent.futures.ThreadPoolExecutor</code>	<code>ExecutorService</code> , <code>ForkJoinPool</code>
GIL	Yes (CPython) — limits true CPU parallelism in threads	No GIL — true CPU parallelism
CPU-bound parallelism	<code>multiprocessing</code> or C extensions	Threads natively
I/O-bound concurrency	Threads, asyncio, or async frameworks	Threads, virtual threads (Java 21)
Synchronization	<code>Lock</code> , <code>RLock</code> , <code>Semaphore</code> , <code>Event</code> , <code>Queue</code>	<code>synchronized</code> , <code>ReentrantLock</code> , <code>ConcurrentHashMap</code>
Async model	<code>asyncio</code> — single-thread event loop	Virtual threads (Loom), reactive (Reactor)
Thread safety	GIL protects some ops, still need locks for compound ops	No GIL — must explicitly synchronize all shared state

2. Python threading — when to use it

Threads share the process memory. The GIL ensures only one runs Python bytecode at a time, so threads are good for I/O-bound work (network, disk, DB) where threads block waiting on I/O and other threads run. Poor for CPU-bound (image processing, numerical computation).

```
from concurrent.futures import ThreadPoolExecutor
import requests

urls = ["https://api/a", "https://api/b", "https://api/c"]

def fetch(url): return requests.get(url).json()

with ThreadPoolExecutor(max_workers=5) as pool:
    futures = [pool.submit(fetch, u) for u in urls]
    results = [f.result() for f in futures]
# All 3 fetches run concurrently, returns in order
```

3. Python multiprocessing — when to use it

Separate processes, separate GILs, true parallel CPU execution. Higher overhead (process spawn, IPC). Use for CPU-bound: data transformation, compression, ML inference, image processing.

```

from concurrent.futures import ProcessPoolExecutor
import os

def transform_chunk(chunk):
    # CPU-intensive: runs in a separate process
    return [row.upper() for row in chunk]

data = [list(range(1000)) for _ in range(8)]

with ProcessPoolExecutor(max_workers=os.cpu_count()) as pool:
    results = list(pool.map(transform_chunk, data))
# Uses all CPU cores; each worker is a real OS process

```

Communication between processes: use `multiprocessing.Queue`, `Pipe`, `shared_memory` (Python 3.8+), or `Manager` for complex objects. Avoid passing large objects through queues — use shared memory or memory-mapped files.

4. asyncio — the third model

Single thread, cooperative multitasking. No parallelism at all, but enables high concurrency for I/O by running thousands of coroutines on one thread. No lock overhead, no GIL concern. Use for FastAPI, async DB drivers (asyncpg), Kafka async clients.

```

import asyncio

async def pipeline():
    async with asyncpg.create_pool(DSN) as pool:
        async with pool.acquire() as conn:
            rows = await conn.fetch("SELECT * FROM events")
        return rows

```

5. When to use what — decision matrix

Scenario	Best choice
HTTP requests to external APIs (many concurrent)	asyncio / ThreadPoolExecutor
CPU-bound data transformation, compression	multiprocessing / ProcessPoolExecutor
FastAPI endpoint handling many requests	asyncio (async def endpoints)
Running ML model inference in batch	multiprocessing
Background email/notification sending	Thread pool or Celery task
Producer-consumer pipeline	asyncio.Queue or multiprocessing.Queue

6. How does Java threading differ in practice?

Java threads are OS-level threads; the JVM maps them 1:1. No GIL means true parallelism for CPU-bound work using threads. Java 21's **virtual threads** (Project Loom) provide lightweight threads similar to Python's asyncio — millions of virtual threads blocking on I/O without blocking OS threads, restoring the simplicity of synchronous code at async scale. Python's asyncio requires explicit `async/await` syntax; Java virtual threads let you write plain blocking code.

```
// Java 21: virtual thread – writes like blocking, scales like async
// Thread.ofVirtual().start() -> {
//     String result = httpClient.send(request).body(); // blocks vthread, not OS
// });

# Python equivalent with asyncio:
async def handler():
    result = await http_client.get(url) # explicit await
```

7. Thread vs Process vs Coroutine — memory and overhead comparison

	Thread	Process	Coroutine (async)
Memory per unit	~8MB stack	~50MB+ (own heap)	~1KB
Startup cost	Milliseconds	Tens of ms	Microseconds
Switch cost	OS context switch	OS context switch + IPC	Function call
Max practical count	Hundreds	Tens–Hundreds	Tens of thousands
Shared memory	Yes (with GIL)	No (need IPC)	Yes (single-thread)

6. Synchronization in Python

1. threading.Lock and RLock

```
import threading

counter = 0
lock = threading.Lock()

def increment():
    global counter
    with lock:          # context manager – always released
        counter += 1    # critical section

threads = [threading.Thread(target=increment) for _ in range(1000)]
for t in threads: t.start()
for t in threads: t.join()
print(counter) # always 1000
```

RLock (re-entrant) allows the same thread to acquire the lock multiple times without deadlocking — useful in recursive functions.

2. threading.Event, Condition, Semaphore

```
import threading

ready = threading.Event()

def producer():
    time.sleep(1)
    ready.set() # signal consumers

def consumer():
    ready.wait() # block until set
    process()

# Semaphore — limits concurrent access to a resource
db_semaphore = threading.Semaphore(5) # max 5 concurrent DB connections

def db_query():
    with db_semaphore:
        result = conn.execute(query)
```

3. Thread-safe queue — the recommended way to share data

```
from queue import Queue
import threading

q = Queue(maxsize=100)

def producer():
    for item in data_source:
        q.put(item) # blocks if full
    q.put(None) # sentinel to stop consumers

def consumer():
    while True:
        item = q.get() # blocks if empty
        if item is None: break
        process(item)
        q.task_done()

threading.Thread(target=producer).start()
for _ in range(4): threading.Thread(target=consumer).start()
```

4. Synchronization in asyncio (single-threaded)

```
import asyncio

lock = asyncio.Lock()          # protect a shared resource
semaphore = asyncio.Semaphore(10) # limit concurrent tasks

async def limited_call(url):
    async with semaphore:      # max 10 concurrent HTTP calls
        return await client.get(url)

# asyncio.Queue – async producer-consumer
q = asyncio.Queue()
async def produce():
    for item in items: await q.put(item)
async def consume():
    while True: process(await q.get()); q.task_done()
```

5. Deadlock — how it happens and prevention in Python

```
# Deadlock: thread 1 holds L1 waits for L2; thread 2 holds L2 waits for L1
# Fix 1: always acquire locks in the same order
# Fix 2: use lock timeout
acquired = lock.acquire(timeout=2.0)
if not acquired:
    raise TimeoutError("Could not acquire lock")

# Fix 3: use threading.local() for per-thread state (no sharing, no lock needed)
local = threading.local()
def get_conn():
    if not hasattr(local, "conn"): local.conn = create_conn()
    return local.conn
```

6. How does Python's synchronization compare to Java?

Python	Java equivalent
<code>threading.Lock</code>	<code>ReentrantLock</code>
<code>threading.RLock</code>	<code>ReentrantLock</code> (reentrant by default)
<code>threading.Semaphore</code>	<code>Semaphore</code>
<code>threading.Event</code>	<code>CountDownLatch</code> / <code>CyclicBarrier</code>
<code>queue.Queue</code>	<code>LinkedBlockingQueue</code>
<code>threading.local()</code>	<code>ThreadLocal<T></code>
<code>asyncio.Lock</code>	Virtual thread + lock in Java 21

7. TDD — Unit & Integration Testing

1. What is TDD? Red-Green-Refactor cycle

Test-Driven Development: write a failing test first (Red), write the minimum code to pass it (Green), then refactor without breaking tests (Refactor). Benefits: forces clear API design, prevents over-engineering, catches regressions, acts as living documentation.

2. Unit testing with pytest — full example

```
# src/order_service.py
class OrderService:
    def __init__(self, repo, payment_client):
        self.repo = repo
        self.payment = payment_client

    def place_order(self, user_id, items):
        if not items:
            raise ValueError("Order must have items")
        total = sum(i["price"] * i["qty"] for i in items)
        result = self.payment.charge(user_id, total)
        if not result.success:
            raise PaymentError("Payment failed")
        order = {"user_id": user_id, "items": items, "total": total}
        return self.repo.save(order)
```

```
# tests/test_order_service.py
import pytest
from unittest.mock import MagicMock, patch

@pytest.fixture
def svc():
    repo = MagicMock(); payment = MagicMock()
    payment.charge.return_value = MagicMock(success=True)
    repo.save.return_value = {"id": "ORD-1"}
    return OrderService(repo, payment)

def test_place_order_success(svc):
    items = [{"price": 100, "qty": 2}]
    result = svc.place_order("u1", items)
    assert result["id"] == "ORD-1"
    svc.payment.charge.assert_called_once_with("u1", 200)

def test_empty_items_raises(svc):
    with pytest.raises(ValueError, match="items"):
        svc.place_order("u1", [])

@pytest.mark.parametrize("qty,price,expected", [
    (1, 50, 50), (3, 20, 60), (2, 100, 200)
])
def test_total_calculation(svc, qty, price, expected):
    items = [{"qty": qty, "price": price}]
    svc.place_order("u1", items)
    svc.payment.charge.assert_called_once_with("u1", expected)
```

3. Integration testing with FastAPI + TestClient

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)

def test_create_order_api(monkeypatch):
    monkeypatch.setattr("app.routes.order.order_svc.place_order",
                        lambda *a: {"id": "ORD-1"})
    resp = client.post("/orders", json={
        "user_id": "u1", "items": [{"price": 100, "qty": 1}]}
    assert resp.status_code == 201
    assert resp.json()["id"] == "ORD-1"
```

4. Integration testing with a real database using testcontainers

```
from testcontainers.postgres import PostgresContainer
import pytest, psycopg2

@pytest.fixture(scope="session")
def pg():
    with PostgresContainer("postgres:15") as pg:
        conn = psycopg2.connect(pg.get_connection_url())
        conn.execute("CREATE TABLE orders (id SERIAL, total FLOAT)")
        yield conn

def test_insert_order(pg):
    pg.execute("INSERT INTO orders(total) VALUES(100)")
    rows = pg.execute("SELECT * FROM orders").fetchall()
    assert rows[0][1] == 100
```

5. Mocking, patching, and test fixtures

```
from unittest.mock import patch, Mock, AsyncMock

def test_kafka_producer():
    with patch("app.events.KafkaProducer") as MockProd:
        mock_instance = MockProd.return_value
        publish_order_event({"id": "1"})
        mock_instance.send.assert_called_once()

async def test_async_http_call():
    with patch("app.client.httpx.AsyncClient.get",
               new_callable=AsyncMock) as mock_get:
        mock_get.return_value = Mock(json=lambda: {"ok": True})
        result = await fetch_status()
        assert result["ok"]
```

6. Code coverage and CI integration

```
# Run tests with coverage
# pytest --cov=src --cov-report=html --cov-fail-under=80

# pyproject.toml
# [tool.pytest.ini_options]
# testpaths = ["tests"]
# [tool.coverage.run]
# source = ["src"]
# omit = ["tests/*"]

# GitHub Actions CI snippet:
# - run: pytest --cov=src --cov-fail-under=80
```

TDD for microservices: unit-test business logic with mocked dependencies; integration-test the HTTP layer with TestClient; contract-test service boundaries with pytest-httpserver or Pact. Run all in CI on every push.

8. FastAPI — Deep Dive

1. Why FastAPI? Key strengths vs Flask and Django

Aspect	FastAPI	Flask	Django
Async	Native (ASGI)	Limited (WSGI)	Partial
Validation	Automatic (Pydantic)	Manual	Serializers (DRF)
Docs	Auto Swagger + ReDoc	Manual	Manual
Performance	Very high (I/O-bound)	Good	Good
Type safety	Central to design	Optional	Limited

2. Full production-grade FastAPI app structure

```
src/
├─ main.py           # app factory, lifespan
├─ config.py         # Settings via pydantic-settings
├─ routers/
│  └─ orders.py
│  └─ users.py
├─ services/
│  └─ order_service.py
├─ repositories/
│  └─ order_repo.py
├─ models/           # SQLAlchemy ORM models
├─ schemas/          # Pydantic request/response models
└─ dependencies.py   # Shared deps: DB session, auth
```

3. Pydantic models — validation, serialization, settings

```
from pydantic import BaseModel, EmailStr, Field, validator
from pydantic_settings import BaseSettings

class OrderIn(BaseModel):
    user_id: int = Field(..., gt=0)
    items: list[dict]
    coupon: str | None = None

    @validator("items")
    def items_not_empty(cls, v):
        if not v: raise ValueError("items required")
        return v

class OrderOut(BaseModel):
    id: str; total: float; status: str
    class Config: from_attributes = True # ORM mode

class Settings(BaseSettings):
    db_url: str; kafka_bootstrap: str; jwt_secret: str
    class Config: env_file = ".env"
```

4. Dependency injection pattern — DB session and auth

```
from fastapi import Depends, HTTPException, Security
from fastapi.security import HTTPBearer

bearer = HTTPBearer()

async def get_db():
    async with AsyncSession(engine) as session:
        yield session # auto-closed after request

async def get_current_user(token=Security(bearer)):
    payload = decode_jwt(token.credentials)
    if not payload:
        raise HTTPException(401, "Invalid token")
    return payload["sub"]

@app.post("/orders", response_model=OrderOut, status_code=201)
async def create_order(
    order: OrderIn,
    db: AsyncSession = Depends(get_db),
    user: str = Depends(get_current_user)
):
    return await order_svc.place(db, user, order)
```

5. Background tasks, middleware, lifespan

```
from contextlib import asynccontextmanager
from fastapi import BackgroundTasks

@asynccontextmanager
async def lifespan(app: FastAPI):
    await db_pool.connect()      # startup
    yield
    await db_pool.disconnect()   # shutdown

app = FastAPI(lifespan=lifespan)

@app.middleware("http")
async def add_trace_id(req, call_next):
    req.state.trace_id = uuid4().hex
    resp = await call_next(req)
    resp.headers["X-Trace-Id"] = req.state.trace_id
    return resp

@app.post("/orders")
async def create(order: OrderIn, bg: BackgroundTasks):
    result = await save_order(order)
    bg.add_task(send_confirmation_email, order.user_id)
    return result
```

6. FastAPI for microservices — service-to-service calls

```
import httpx
from functools import lru_cache

@lru_cache
def get_settings(): return Settings()

async def call_inventory(item_id: int):
    settings = get_settings()
    async with httpx.AsyncClient() as c:
        r = await c.get(
            f"{settings.inventory_url}/items/{item_id}",
            headers={"X-Service-Token": settings.service_token},
            timeout=3.0)
        r.raise_for_status()
    return r.json()
```

9. REST API

1. REST principles and HTTP semantics

Stateless, resource-based URIs, uniform interface (GET/POST/PUT/PATCH/DELETE), representations (JSON), HATEOAS (links in responses). GET=read(safe+idempotent), PUT=full replace (idempotent), PATCH=partial update, DELETE=remove (idempotent), POST=create/action (not idempotent).

2. Request headers critical for APIs

Header	Use
Content-Type: application/json	Body format being sent
Accept: application/json	Desired response format
Authorization: Bearer <jwt>	Auth token
X-Request-ID	Distributed tracing, idempotency
Cache-Control: no-cache	Caching directive
If-None-Match: <etag>	Conditional GET (304 Not Modified)

3. API versioning strategies

```
# URI versioning (most common)
@app.get("/v1/orders/{id}")
@app.get("/v2/orders/{id}")

# Header versioning
# Accept: application/vnd.company.v2+json

# Query param
# GET /orders/1?version=2
```

4. Authentication vs Authorization (implementation)

Authentication = identity proof (JWT validation, OAuth2, API key check → 401 on failure). **Authorization** = permission check after identity is known (RBAC, scopes → 403 on failure). In FastAPI: auth in a dependency, authz via `@PreAuthorize` equivalent or role check in the dependency.

```
async def require_admin(user=Depends(get_current_user)):
    if user.role != "admin":
        raise HTTPException(403, "Admin only")
    return user
```

5. Status code quick reference (for experience candidates)

Code	Meaning	When to use
200	OK	Successful GET/PUT/PATCH
201	Created	POST creating a resource
204	No Content	DELETE success
400	Bad Request	Malformed syntax
401	Unauthorized	Missing/invalid credentials
403	Forbidden	Authed but no permission
404	Not Found	Resource missing
409	Conflict	Duplicate resource
422	Unprocessable	Valid syntax, fails validation
429	Too Many Requests	Rate limited
500	Internal Server Error	Unhandled server bug
503	Service Unavailable	Overloaded or circuit open

10. Apache Camel

1. What is Apache Camel and what problem does it solve?

Apache Camel is an open-source integration framework implementing Enterprise Integration Patterns (EIPs). It provides a routing and mediation engine that connects heterogeneous systems (REST, Kafka, FTP, database, S3, SAP, etc.) using a consistent DSL. It solves the "integration spaghetti" problem — instead of writing bespoke glue code for every system pair, you define routes in Camel's DSL and it handles protocols, transformations, error handling, and retries.

2. Core concepts: Route, Processor, Component, Exchange

Concept	What it is
Route	A definition of message flow from one endpoint to another (<code>from(...).to(...)</code>)
Exchange	The message container — holds in/out Message, headers, and properties
Message	Actual payload + headers + attachments
Processor	A step that processes/transforms the Exchange
Component	A connector/adaptor (kafka, http, file, s3, jdbc, etc.)
Endpoint	A specific address on a Component (e.g. <code>kafka:my-topic</code>)

3. Camel route examples (Java DSL)

```
// Consume from Kafka → transform → POST to REST API
// from("kafka:orders?brokers=localhost:9092")
//   .unmarshal().json(OrderDto.class)
//   .process(exchange -> {
//       OrderDto dto = exchange.getIn().getBody(OrderDto.class);
//       exchange.getIn().setBody(enrichOrder(dto));
//   })
//   .marshal().json()
//   .to("https://api.company.com/orders");

// File to DB — EIP: Splitter + Aggregate
// from("file:inbox?delete=true")
//   .split(body().tokenize("\n")).streaming()
//   .process(this::parseCsvRow)
//   .to("jdbc:dataSource?useHeadersAsParameters=true");
```

4. Common EIPs implemented by Camel

- **Splitter** — splits one message into many (e.g., batch CSV → individual rows).
- **Aggregator** — collects many messages into one (e.g., batch DB inserts).
- **Content-Based Router** — routes messages to different destinations based on content (`.choice().when(header("type").is("order")).to(...)`).
- **Dead Letter Channel** — sends failed messages to a DLQ after retries.
- **Idempotent Consumer** — deduplicates messages using an ID store.
- **Filter** — drops messages not matching a condition.
- **Enrich** — fetches additional data to augment a message.

5. Camel error handling and retry

```
// errorHandler(deadLetterChannel("kafka:failed-orders")
//   .maximumRedeliveries(3)
//   .redeliveryDelay(1000)
//   .backOffMultiplier(2.0)
//   .retryAttemptedLogLevel(LoggingLevel.WARN));
```

6. Where does Camel fit vs Kafka?

Kafka is an event streaming platform — durable, scalable pub/sub. **Camel** is an integration framework — routing, transformation, protocol bridging. They complement: Camel routes often consume from Kafka, transform, and produce to another system. Camel has a native Kafka component; Kafka has no concept of routing logic — that's Camel's job.

11. Apache Kafka

1. Kafka core architecture

A distributed commit log. **Topics** are partitioned and replicated across **brokers**. **Producers** append records; **consumers** (in groups) read at their own pace tracking **offsets**. Records are retained for a configurable period — consumers can replay. **ZooKeeper** (legacy) or **KRaft** (Kafka 3+) handles cluster metadata.

2. Producing and consuming with Python (confluent-kafka)

```
from confluent_kafka import Producer, Consumer, KafkaError
import json

# Producer
producer = Producer({"bootstrap.servers":"localhost:9092"})

def publish(topic, key, value):
    producer.produce(
        topic,
        key=str(key).encode(),
        value=json.dumps(value).encode(),
        callback=lambda err, msg: (
            print(f"Delivered {msg.offset()}") if not err
            else print(f"Error: {err}")
        )
    )
    producer.flush()

publish("orders", "ORD-1", {"user":"u1", "total":200})
```

```
# Consumer
consumer = Consumer({
    "bootstrap.servers":"localhost:9092",
    "group.id":"order-processor",
    "auto.offset.reset":"earliest",
    "enable.auto.commit":False # manual commit
})
consumer.subscribe(["orders"])
try:
    while True:
        msg = consumer.poll(1.0)
        if msg is None or msg.error(): continue
        order = json.loads(msg.value())
        process_order(order)
        consumer.commit(asynchronous=False) # commit after processing
finally:
    consumer.close()
```

Manual commit after processing ensures at-least-once delivery — if the consumer crashes before committing, the message is re-read on restart. Design your consumers to be idempotent.

3. Partitioning, offsets, and ordering guarantees

Partitioning: a topic is split into N partitions for parallelism. A key-based partition strategy routes all records with the same key to the same partition — guaranteeing ordering per key. An offset is the sequential ID within a partition. With a consumer group of N consumers, Kafka assigns at most one consumer per partition.

4. Delivery semantics

Semantic	Config	Risk
At-most-once	Auto-commit before processing	Message loss on crash
At-least-once	Manual commit after processing	Duplicate processing
Exactly-once	<code>enable.idempotence=true</code> + transactional API	Highest overhead

5. Dead letter queue pattern for failed messages

```
def safe_process(msg):
    try:
        process_order(json.loads(msg.value()))
        consumer.commit(asynchronous=False)
    except Exception as e:
        logging.error(f"Failed processing {msg.offset()}: {e}")
        # Send to DLQ topic instead of stopping
        producer.produce("orders-dlq", msg.value(),
                        headers={"error": str(e).encode()})
        producer.flush()
        consumer.commit(asynchronous=False) # move past bad message
```

12. Couchbase

1. What is Couchbase and when to use it?

Couchbase is a distributed, multi-model NoSQL database combining a key-value store with a JSON document store, full SQL++ query language (N1QL), full-text search, analytics, and an optional cache/eviction mode. Use cases: session management (sub-millisecond reads), user profiles, product catalogs, IoT telemetry, and anywhere you need flexible JSON storage at scale with strong read/write performance.

2. Connecting with Python SDK

```
from couchbase.cluster import Cluster
from couchbase.auth import PasswordAuthenticator
from couchbase.options import ClusterOptions, GetOptions
from datetime import timedelta

cluster = Cluster("couchbase://localhost",
                 ClusterOptions(PasswordAuthenticator("user", "pass")))
cluster.wait_until_ready(timedelta(seconds=5))
bucket = cluster.bucket("travel-sample")
coll = bucket.default_collection()
```

3. CRUD operations

```
# Insert
coll.insert("order::1001", {"user": "u1", "total": 200, "status": "pending"})

# Get with timeout
result = coll.get("order::1001", GetOptions(timeout=timedelta(seconds=2)))
print(result.content_as[dict])

# Upsert (insert or replace)
coll.upsert("order::1001", {"total": 250, "status": "paid"})

# Sub-document update (efficient partial update)
coll.mutate_in("order::1001", [SD.upsert("status", "shipped")])

# Delete
coll.remove("order::1001")
```

4. N1QL queries (SQL-like for JSON documents)

```
# N1QL query
result = cluster.query(
    """
    SELECT u.id, u.name, COUNT(o.id) AS order_count
    FROM users u
    JOIN orders o ON KEYS o.user_id
    WHERE u.region = $region
    GROUP BY u.id, u.name
    HAVING COUNT(o.id) > 5
    """,
    QueryOptions(named_parameters={"region":"WEST"})
)
for row in result: print(row)
```

5. Couchbase vs Redis vs Oracle DB — when to use which

Use Case	Best Choice
Fast sessions / pure cache	Redis (sub-ms, simple)
JSON docs + SQL queries + scale	Couchbase
Complex joins, ACID, reporting	Oracle DB / PostgreSQL
Time-series IoT data	Couchbase or TimescaleDB
Full-text search on docs	Couchbase FTS or Elasticsearch

6. Couchbase TTL and expiry — ideal for sessions

```
from couchbase.options import InsertOptions

# Document auto-expires after 30 minutes
coll.insert(f"session::{token}",
    {"user_id": uid, "ip": ip_addr},
    InsertOptions(expiry=timedelta(minutes=30)))
```

13. Airflow DAGs & Tasks

1. What is Airflow and what is a DAG?

Apache Airflow is a workflow orchestration platform. A DAG (Directed Acyclic Graph) defines a workflow: tasks (nodes) and dependencies (edges), with no cycles. Airflow schedules runs, tracks state, handles retries, and provides a UI for monitoring.

2. Creating a DAG — classic vs TaskFlow API

```
# Classic style
from airflow import DAG
from airflow.operators.python import PythonOperator
from airflow.providers.amazon.aws.operators.s3 import S3CreateObjectOperator
from datetime import datetime, timedelta

default_args = {
    "owner": "data-team", "retries": 2,
    "retry_delay": timedelta(minutes=5),
    "email_on_failure": True
}

def extract(**ctx): return fetch_from_api(ctx["ds"])
def transform(ti, **ctx):
    data = ti.xcom_pull(task_ids="extract")
    return [clean(row) for row in data]

with DAG("order_etl", schedule="@daily",
        start_date=datetime(2024,1,1),
        default_args=default_args,
        catchup=False) as dag:

    t_extract = PythonOperator(task_id="extract", python_callable=extract)
    t_transform = PythonOperator(task_id="transform", python_callable=transform)

    t_extract >> t_transform # dependency: transform runs after extract
```

```
# Modern TaskFlow API (Airflow 2.0+) – much cleaner
from airflow.decorators import dag, task

@dag(schedule="@daily", start_date=datetime(2024,1,1), catchup=False)
def order_pipeline():

    @task
    def extract(ds=None):
        return fetch_from_api(ds) # XCom automatically

    @task
    def transform(raw_data):
        return [clean(r) for r in raw_data]

    @task
    def load(clean_data):
        db_bulk_insert(clean_data)

    load(transform(extract())) # Python call order = dependency

order_pipeline()
```

3. AWS operators in Airflow (MWAA context)

```
from airflow.providers.amazon.aws.operators.glue import GlueJobOperator
from airflow.providers.amazon.aws.operators.lambda_function import LambdaInvokeFunctionOperator
from airflow.providers.amazon.aws.sensors.s3 import S3KeySensor

with DAG("aws_pipeline", ...) as dag:
    wait_for_file = S3KeySensor(
        task_id="wait", bucket_name="data-lake",
        bucket_key="input/{{ ds }}/orders.csv",
        mode="reschedule")    # frees worker slot while waiting

    run_glue = GlueJobOperator(
        task_id="glue_transform", job_name="OrderTransform",
        script_args={"--date": "{{ ds }}"})

    invoke_lambda = LambdaInvokeFunctionOperator(
        task_id="notify", function_name="send-report",
        payload=b'{"date": "{{ ds }}"}')

    wait_for_file >> run_glue >> invoke_lambda
```

4. XComs — passing data between tasks

XComs ("cross-communications") push/pull small values between tasks via Airflow's metadata DB. Use for task output references, not large datasets — store large data in S3 and pass the S3 key via XCom.

```
def save_to_s3(ti, **ctx):
    path = upload_to_s3(data, f"s3://bucket/{ctx['ds']}/data.json")
    ti.xcom_push(key="s3_path", value=path)

def process(ti):
    path = ti.xcom_pull(task_ids="save_to_s3", key="s3_path")
    data = read_from_s3(path)
```

14. AWS Lambda & Orchestration

1. How is orchestration done using AWS Lambda and Airflow?

Two patterns: (a) **Airflow triggers Lambda** using `LambdaInvokeFunctionOperator` — Airflow owns the orchestration schedule and dependencies, Lambda executes individual lightweight tasks (sending notifications, small transforms). (b) **Lambda triggers Airflow DAG runs** via the Airflow REST API — an S3 event or SQS message fires a Lambda that POSTs to MWAA's REST endpoint to start a DAG, useful for event-driven pipelines.

```
# Lambda triggering an MWSA DAG run (pattern b)
import boto3, json, requests

def handler(event, ctx):
    mwaa = boto3.client("mwaa")
    token = mwaa.create_cli_token(Name="my-mwaa-env")
    resp = requests.post(
        f"https://{token['WebServerHostname']}/aws_mwaa/cli",
        headers={"Authorization": f"Bearer {token['CliToken']}"},
        json={"command": f"dags trigger order_pipeline --conf '{json.dumps(event)}'"})
    )
```

2. Python Lambda handler — best practices

```
import json, logging, boto3
from typing import Any

logger = logging.getLogger()
logger.setLevel(logging.INFO)

# Initialize outside handler — reused across warm invocations
s3 = boto3.client("s3")
db_conn = create_db_connection()

def handler(event: dict, context: Any) -> dict:
    logger.info(f"RequestId: {context.aws_request_id}")
    try:
        records = event.get("Records", [])
        results = [process_record(r) for r in records]
        return {"statusCode": 200, "body": json.dumps({"processed": len(results)})}
    except Exception as e:
        logger.exception("Unhandled error")
        raise # Lambda retries on raise; DLQ captures after max retries
```

15. Streaming Large Files in AWS Lambda

How do you stream while processing large files in AWS Lambda?

Lambda has a 512 MB (up to 10 GB configurable) ephemeral `/tmp`. For large files, **never load them fully into memory** — stream from S3 using `StreamingBody`.

```

import boto3, csv, io

s3 = boto3.client("s3")

def handler(event, ctx):
    bucket = event["bucket"]; key = event["key"]

    # Stream from S3 — does NOT load whole file
    response = s3.get_object(Bucket=bucket, Key=key)
    stream = response["Body"] # StreamingBody

    # Wrap in text mode and process line by line
    reader = csv.DictReader(io.TextIOWrapper(stream, encoding="utf-8"))
    processed = 0
    batch = []

    for row in reader: # one row in memory at a time
        batch.append(transform(row))
        if len(batch) >= 500: # batch write every 500 rows
            db_bulk_insert(batch)
            batch.clear()
            processed += 500

    if batch: db_bulk_insert(batch) # flush remainder
    return {"processed": processed + len(batch)}

```

Pattern: multipart upload + chunked processing for huge files

```

# For files that exceed Lambda's memory limit or timeout (15 min),
# use byte-range reads and process in chunks:
def stream_s3_in_chunks(bucket, key, chunk_bytes=50*1024*1024):
    head = s3.head_object(Bucket=bucket, Key=key)
    size = head["ContentLength"]
    offset = 0
    while offset < size:
        end = min(offset + chunk_bytes - 1, size - 1)
        chunk = s3.get_object(
            Bucket=bucket, Key=key,
            Range=f"bytes={offset}-{end}")["Body"].read()
        yield chunk
        offset = end + 1

# Use SQS + Lambda for each chunk for true parallelism on very large files

```

For very large files (>1 GB): split into chunks with S3 Select, Glue, or EMR. Lambda is best for small-to-medium files (<200 MB) or event-driven per-record processing. Use Step Functions to orchestrate multiple Lambda chunk-processors in parallel.

16. Java/Scala → Python Migration

What would be your approach to migrating a Java/Scala application to Python?

Phase 1 — Discovery & Analysis: inventory all modules, dependencies, integrations, and performance SLAs. Classify as: pure business logic (easy to port), framework-heavy (Spring → FastAPI), Spark/Scala (→ PySpark), or C-extension-dependent (may stay as microservice).

Phase 2 — Architecture decisions:

- Spring Boot REST → FastAPI (async, Pydantic validation).
- Hibernate ORM → SQLAlchemy (async).
- Spring Security JWT → python-jose + FastAPI dependencies.
- Kafka (Spring) → confluent-kafka-python.
- Scala Spark jobs → PySpark (often a near 1:1 translation of transformations).
- JUnit/Mockito tests → pytest + unittest.mock.

Phase 3 — Incremental migration (Strangler Fig pattern): replace one service/module at a time behind the same API contract. Run both versions in parallel behind a proxy; shift traffic gradually. Never big-bang rewrite.

Phase 4 — Key watch-outs:

- Java is statically typed at compile time — add mypy + type hints to Python equivalent.
- Java threads map 1:1 to OS threads; Python uses asyncio or multiprocessing — redesign concurrency model.
- Java generics → Python TypeVar/Generic.
- Scala case classes → Python dataclasses or Pydantic models.
- Maven/Gradle → pip + pyproject.toml (Poetry).
- Performance benchmarking before and after for each migrated component.

17. Terraform & Databricks

1. What is Terraform?

An Infrastructure as Code (IaC) tool by HashiCorp that provisions and manages cloud resources declaratively using HCL (HashiCorp Configuration Language). It tracks state, generates execution plans (`terraform plan`), and applies changes (`terraform apply`). Supports AWS, Azure, GCP, Databricks, and hundreds of providers.

```
# AWS Lambda + SQS trigger in Terraform
# resource "aws_lambda_function" "processor" {
#   function_name = "order-processor"
#   runtime       = "python3.12"
#   handler       = "main.handler"
#   filename      = "build/lambda.zip"
#   role          = aws_iam_role.lambda_exec.arn
#   environment { variables = { KAFKA_BOOTSTRAP = var.kafka_url } }
# }
#
# resource "aws_lambda_event_source_mapping" "sqs_trigger" {
#   event_source_arn = aws_sqs_queue.orders.arn
#   function_name    = aws_lambda_function.processor.arn
#   batch_size       = 10
# }
```

2. Terraform state management best practices

Store state remotely in S3 (with DynamoDB locking) — never locally in a team. Use workspaces for environment isolation (dev/staging/prod). Lock state during apply. Use `terraform import` to bring existing resources under management. Use modules for reusable components.

3. What is Databricks?

A unified data analytics platform built on Apache Spark. Provides managed Spark clusters, notebooks (Python/Scala/SQL/R), Delta Lake (ACID transactions on data lakes), MLflow for ML lifecycle, Delta Live Tables for streaming ETL, and Unity Catalog for data governance. Runs on AWS/Azure/GCP.

4. Running Python on Databricks

```
# Databricks notebook / job (Python)
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, to_date

spark = SparkSession.builder.getOrCreate()

df = spark.read.format("delta").load("s3://datalake/orders/")

result = (df
    .filter(col("status") == "completed")
    .withColumn("order_date", to_date(col("created_at")))
    .groupBy("order_date", "region")
    .sum("total")
)
result.write.format("delta").mode("overwrite").save("s3://datalake/daily_revenue/")

# Databricks Jobs API – trigger programmatically from Lambda or Airflow
```

5. Databricks + Airflow integration

```
from airflow.providers.databricks.operators.databricks import DatabricksRunNowOperator

run_notebook = DatabricksRunNowOperator(
    task_id="run_etl",
    databricks_conn_id="databricks_default",
    job_id="1234",
    notebook_params={"date": "{{ ds }}"})
```

18. Microservices & Team Leadership

1. Design a Python microservices system

Each service: its own FastAPI app, its own DB (PostgreSQL/Couchbase per need), containerized in Docker, communicating via REST (sync) and Kafka (async events). API Gateway (Kong/AWS) at the edge for auth, routing, rate limiting. Services discoverable via Kubernetes DNS or Consul. Observability: Prometheus + Grafana metrics, ELK logs, Jaeger tracing with a shared trace-ID header.

2. How would you lead and guide a Python team?

Set coding standards (PEP 8, type hints, mypy in CI), enforce TDD with minimum 80% coverage, peer code reviews with a checklist (error handling, tests, types, security, performance). Document ADRs (Architecture Decision Records) for key decisions. Mentor via pair programming and tech talks. Implement a clear branching strategy (trunk-based or gitflow), feature flags for safe releases, and blameless post-mortems for production incidents.

3. How do you approach introducing TDD in an existing Python codebase?

Start with the new code — any new feature gets tests written first. Gradually add tests for existing code when changing it (Boy Scout Rule). Set a coverage floor and increase it each sprint. Use characterization tests for legacy code to document current behavior before refactoring. Share the feedback loop in standups — green CI is celebrated, broken tests stop the line.

19. Coding Challenge: Anagram Grouping

Problem: Group words that are anagrams of each other

Input: `["eat", "tea", "tan", "ate", "nat", "bat"]` → Output:

`[["eat", "tea", "ate"], ["tan", "nat"], ["bat"]]`

Key insight: two words are anagrams iff sorting their characters gives the same string. Use that as the hash map key.

```
# Approach 1: Sort-based key — O(n * k log k)
from collections import defaultdict

def group_anagrams(words):
    groups = defaultdict(list)
    for word in words:
        key = tuple(sorted(word)) # "eat"→('a','e','t')
        groups[key].append(word)
    return list(groups.values())

print(group_anagrams(["eat", "tea", "tan", "ate", "nat", "bat"]))
# [['eat', 'tea', 'ate'], ['tan', 'nat'], ['bat']]
```

```
# Approach 2: Character frequency key — O(n * k), no sorting
def group_anagrams_freq(words):
    groups = defaultdict(list)
    for word in words:
        freq = [0] * 26
        for ch in word:
            freq[ord(ch) - ord('a')] += 1
        groups[tuple(freq)].append(word)
    return list(groups.values())
```

```
# Approach 3: One-liner using itertools.groupby
from itertools import groupby

def group_anagrams_functional(words):
    keyfn = lambda w: sorted(w)
    return [list(g) for _, g in groupby(sorted(words, key=keyfn), key=keyfn)]
```

Complexity: Approach 1: $O(n \cdot k \log k)$ time, $O(n \cdot k)$ space. Approach 2: $O(n \cdot k)$ time, $O(n \cdot k)$ space — faster for very long words. Always mention the trade-off.

```
# Reverse string without slicing – O(n) two-pointer
def reverse_string(s):
    chars = list(s); l, r = 0, len(chars)-1
    while l < r:
        chars[l], chars[r] = chars[r], chars[l]
        l += 1; r -= 1
    return "".join(chars)

print(reverse_string("hello")) # olleh
```

20. Quick Reference — Common Pitfalls & Interview Tips

Topic	Common Pitfall	Correct approach
Mutable defaults	<code>def f(x=[]):</code>	<code>def f(x=None): x = x or []</code>
Thread safety	Assuming GIL protects compound ops	Use Lock for read-modify-write
Async in Lambda	Mixing sync and async without <code>asyncio.run</code>	<code>asyncio.run(main())</code> in handler
Kafka offset commit	Auto-commit before processing	Manual commit after successful processing
Memory in Lambda	Loading full S3 file into memory	Stream with <code>StreamingBody</code>
Exception handling	Bare <code>except:</code>	Catch specific exception types
TDD order	Writing code first, tests after	Test first (Red-Green-Refactor)
Couchbase large result	N1QL <code>SELECT *</code> without LIMIT	Paginate with LIMIT/OFFSET or keyset
FastAPI sync in async	Blocking call in <code>async def</code>	Use <code>def</code> (threadpool) or await async equivalent

Final interview tip for the lead role: Frame every answer in the context of a team — "I would set a standard that...", "I'd ensure the team tests this with...", "when guiding a junior, I'd walk through...". The hiring requirement explicitly says "lead and guide a team" — showing engineering leadership mindset on every answer separates a senior individual contributor from a tech lead.